Folleto del estudiante Encuentro 2: Acceso a bases de datos y programación concurrente

Departamento de Ciencias, Tecnología y Salud – CUR Matagalpa

Fecha: ___/___/2026

UNIVERSIDAD NACIONAL AUTÓNOMA DE NICARAGUA, MANAGUA
UNAN-MANAGUA

UNIVERSIDAD EN EL campo

Universidad Nacional Autónoma de Nicaragua, Managua · Centro Universitario Regional de Matagalpa · Departamento de Ciencias, Tecnología y Salud

Programación de Aplicaciones de Escritorio – Folleto del Encuentro 2: Acceso a bases de datos y programación concurrente

Heydy Judelki Mairena Palacios

Ingeniería en Sistemas · II año – II semestre 2026 · Unidad(es) II, III del programa

Fecha de emisión: ___/___/2026

¡Universidad del Pueblo y para el Pueblo!

Elaborado por:	Revisado por:	Autorizado por:
Heydy Judelki Mairena Palacios	⚠ PENDIENTE (coordinador/a de carrera)	⚠ PENDIENTE (director/a de departamento)
Perfil del autor (opcional): ⚠ completar si se desea.		

Presentación

En este módulo AgroStock deja de olvidar sus datos al cerrarse: los guardaremos en una base de datos SQLite y realizaremos las operaciones CRUD desde la interfaz. Luego aprenderemos a ejecutar procesos largos (como importar cientos de productos) en segundo plano, para que la ventana nunca diga «No responde». Se abordan las Unidades II y III del programa.

Proyecto integrador: AgroStock: aplicación de escritorio para el inventario de un agroservicio o cooperativa de la zona (productos, categorías, existencias, reportes y tipo de cambio). Se construye por etapas: v1 interfaz gráfica (E1) → v2 base de datos SQLite + importación concurrente (E2) → v3 ORM, reportes PDF/Excel y consumo de servicio web (E3).

Horas del programa para este módulo: HT 10 · HP 20 · HTI 15 (total 45 h).

Competencias y resultados de aprendizaje

Competencias del programa a las que contribuye este módulo:

- Desarrolla aplicaciones de escritorio con interfaz gráfica que acceden a bases de datos, implementan programación concurrente (hilos), utilizan ORM, generan reportes y consumen servicios web; mediante la aplicación de un lenguaje de programación orientado a objetos para resolver problemas de la vida cotidiana.
- Aplica procesos creativos e investigativos haciendo uso de medios tecnológicos en el desarrollo de sus aprendizajes.
- Demuestra actitudes de respeto hacia las personas y los saberes ancestrales, inclusión y equidad para una cultura de paz en su entorno.

Resultados de aprendizaje:

- Desarrolla aplicaciones de escritorio con interfaz gráfica que acceden a bases de datos y realizan operaciones CRUD (Create, Read, Update, Delete).
- Desarrolla aplicaciones de escritorio con interfaz gráfica que implementan programación concurrente.

Objetivos del encuentro:

- Conectar una aplicación de escritorio a una base de datos SQLite y realizar operaciones CRUD mediante SQL parametrizado y clases de acceso a datos.
- Implementar operaciones en segundo plano con hilos y tareas, sincronización y detención controlada, manteniendo la interfaz responsiva.

Contenidos

Unidad II. Acceso a bases de datos: Introducción a la programación con bases de datos; El lenguaje SQL; Clases para la gestión de bases de datos; Controles de acceso a bases de datos; Operaciones CRUD sobre una base de datos.

Unidad III. Programación concurrente: Introducción a la programación concurrente; Clases para trabajar con hilos; Estados de un hilo; Mecanismos de sincronización; Detención controlada de hilos.

I. Entrada temática – Vivencia

El viernes, en el agroservicio de doña Martha, se fue la luz a media mañana. Cuando volvió, AgroStock v1 abrió vacío: todo lo registrado estaba en memoria. El lunes, su hijo quiso cargar la lista de 500 productos que le envió el distribuidor y la ventana se quedó blanca, con el mensaje «No responde», durante casi un minuto.

1. ¿Dónde deberían guardarse los datos para que sobrevivan a un apagón?
2. Si dos personas registran ventas a la vez, ¿qué problemas pueden aparecer?
3. ¿Por qué crees que la ventana «no responde» mientras se importa la lista?
4. ¿Qué haría un buen programa mientras realiza una tarea larga? Piensa en las aplicaciones que usas en el celular.

> Lo que esperamos aprender: guardar y consultar datos en una base relacional desde C#, y ejecutar tareas largas en segundo plano de forma segura y cancelable.

II. Desarrollo central – Fundamentación científica

Unidad II · 1. Introducción a la programación con bases de datos

Una base de datos relacional organiza la información en tablas formadas por filas (registros) y columnas (campos). Cada tabla tiene una clave primaria que identifica cada fila, y las relaciones entre tablas se expresan con claves foráneas. El sistema gestor (SGBD) garantiza la integridad de los datos mediante restricciones.

En este curso usamos SQLite: un motor relacional que guarda toda la base en un único archivo (agrostock.db) y no requiere instalar un servidor, lo que lo hace ideal para aplicaciones de escritorio y equipos con recursos limitados. Los conceptos y el código son equivalentes para MySQL o SQL Server: solo cambia el proveedor (MySqlConnection, SqlConnection).

Tabla	Campos principales	Relación
categorias	id (PK), nombre (UNIQUE)	Una categoría tiene muchos productos
productos	id (PK), codigo (UNIQUE), nombre, categoria_id (FK), precio (CHECK > 0), existencia, vencimiento	Pertenece a una categoría
movimientos	id (PK), producto_id (FK), tipo (ENTRADA/SALIDA), cantidad, fecha	Muchos movimientos por producto

Arquitectura: la aplicación no habla directamente con el archivo. Usa un proveedor ADO.NET (Microsoft.Data.Sqlite) que traduce las llamadas de C# en instrucciones para el motor.

```
Formulario (UI)  →  ProductoRepository  →  Microsoft.Data.Sqlite  →  agrostock.db
  eventos           clases de acceso        proveedor ADO.NET        archivo SQLite
```

Unidad II · 2. El lenguaje SQL

SQL (Structured Query Language) es el lenguaje estándar de las bases relacionales. Se divide en DDL (definición: CREATE, ALTER, DROP) y DML (manipulación: INSERT, SELECT, UPDATE, DELETE).

```
CREATE TABLE productos (
    id           INTEGER PRIMARY KEY AUTOINCREMENT,
    codigo       TEXT    NOT NULL UNIQUE,
    nombre       TEXT    NOT NULL,
    categoria_id INTEGER NOT NULL REFERENCES categorias(id),
    precio       REAL    NOT NULL CHECK (precio > 0),
    existencia   INTEGER NOT NULL DEFAULT 0 CHECK (existencia >= 0),
    vencimiento  TEXT    NULL
);
```

Operación CRUD	Sentencia SQL	Ejemplo
Create	INSERT	INSERT INTO productos (codigo, nombre, categoria_id, precio) VALUES ('FER-004','Sulfato',1,980);
Read	SELECT	SELECT codigo, nombre FROM productos WHERE existencia < 10 ORDER BY nombre;
Update	UPDATE	UPDATE productos SET precio = precio * 1.05 WHERE categoria_id = 1;
Delete	DELETE	DELETE FROM productos WHERE codigo = 'HER-005';

```
-- JOIN: nombre de la categoría junto al producto
SELECT p.codigo, p.nombre, c.nombre AS categoria
FROM productos p JOIN categorias c ON c.id = p.categoria_id;

-- GROUP BY: valor del inventario por categoría
SELECT c.nombre, COUNT(*) AS productos, SUM(p.precio * p.existencia) AS valor
FROM productos p JOIN categorias c ON c.id = p.categoria_id
GROUP BY c.nombre ORDER BY valor DESC;
```

> Cuidado: un UPDATE o DELETE sin WHERE afecta a todas las filas de la tabla. Escribe primero el SELECT con el mismo WHERE para ver qué filas cambiarán.

Unidad II · 3. Clases para la gestión de bases de datos

ADO.NET define un conjunto de clases comunes a todos los proveedores. Con Microsoft.Data.Sqlite se llaman así:

Clase	Responsabilidad
SqliteConnection	Abre y cierra la conexión con el archivo de la base (cadena de conexión).
SqliteCommand	Contiene la sentencia SQL y sus parámetros. ExecuteReader (SELECT), ExecuteNonQuery (INSERT/UPDATE/DELETE), ExecuteScalar (un valor).
SqliteDataReader	Recorre las filas del resultado, hacia adelante y de solo lectura.
SqliteTransaction	Agrupa varias operaciones: se confirman todas (Commit) o ninguna (Rollback).

```
public List<Producto> Listar(string filtro = "")
{
    using var cn = Db.Abrir();                // using: cierra la conexión siempre
    using var cmd = cn.CreateCommand();
    cmd.CommandText = SelectBase + " WHERE p.nombre LIKE $f OR p.codigo LIKE $f ORDER BY
p.nombre";
    cmd.Parameters.AddWithValue("$f", $"%{filtro}%");   // parámetro, nunca concatenar

    var lista = new List<Producto>();
    using var rd = cmd.ExecuteReader();
    while (rd.Read())
        lista.Add(new Producto { Id = rd.GetInt32(0), Codigo = rd.GetString(1), Nombre =
rd.GetString(2) /* ... */ });
    return lista;
}
```

Inyección SQL: si se concatena lo que escribe el usuario ("... WHERE nombre = '" + txtBuscar.Text + "'"), alguien puede escribir ' OR 1=1 -- y alterar la consulta. Los parámetros ($f) envían el valor por separado y el motor nunca lo interpreta como código SQL.

Patrón Repositorio: todo el SQL vive en una clase (ProductoRepository) y los formularios solo llaman a sus métodos (Listar, Insertar, Actualizar, Eliminar). Así la interfaz no depende del motor de base de datos.

Unidad II · 4. Controles de acceso a bases de datos y CRUD

Windows Forms enlaza datos a controles mediante la propiedad DataSource. BindingSource actúa como intermediario: conoce el elemento actual (Current), permite filtrar y notifica cambios.

```
fuente.DataSource = repo.Listar(txtBuscar.Text);   // BindingSource
dgv.DataSource = fuente;                           // DataGridView

cboCategoria.DataSource = repo.Categorias();
cboCategoria.DisplayMember = "Nombre";   // lo que se ve
cboCategoria.ValueMember = "Id";         // lo que se guarda (SelectedValue)

private void Nuevo()
{
    var p = new Producto();
    using var dlg = new FrmProducto(p, repo.Categorias());
    if (dlg.ShowDialog(this) != DialogResult.OK) return;
    try { repo.Insertar(p); Cargar(); }
    catch (SqliteException ex) when (ex.SqliteErrorCode == 19)   // SQLITE_CONSTRAINT
    {
        MessageBox.Show(this, $"El código {p.Codigo} ya existe.", "No se guardó");
    }
}
```

Unidad III · 5. Introducción a la programación concurrente

Un proceso es un programa en ejecución con su propia memoria. Un hilo (thread) es una secuencia de instrucciones dentro de un proceso; un proceso puede tener varios hilos que comparten la misma memoria. La programación concurrente consiste en organizar varias tareas que progresan al mismo tiempo, ya sea en paralelo (varios núcleos) o intercalándose.

Las aplicaciones Windows Forms tienen un único hilo de interfaz (UI thread) que procesa los eventos y dibuja la ventana. Si un manejador de eventos tarda 30 segundos, durante ese tiempo la ventana

no puede redibujarse ni responder: Windows la marca como «No responde». La solución es mover el trabajo pesado a otro hilo y devolver solo los resultados a la interfaz.

Unidad III · 6. Clases para trabajar con hilos

Clase / técnica	Cuándo usarla
Thread	Control directo de un hilo (nombre, prioridad, IsBackground). Útil para aprender los estados; costoso crear muchos.
ThreadPool	Conjunto de hilos reutilizables administrado por .NET.
Task + Task.Run	Opción recomendada: ejecuta trabajo en el ThreadPool y se puede esperar con await.
async / await	Espera una tarea sin bloquear el hilo de la interfaz; el código continúa en la UI al terminar.
BackgroundWorker	Componente histórico de Windows Forms (eventos DoWork, ProgressChanged, RunWorkerCompleted).
IProgress<T> / Progress<T>	Reporta avance desde el hilo de trabajo; el manejador se ejecuta en el hilo de la UI.

```
btnSegundoPlano.Click += async (_, _) =>
{
    cts = new CancellationTokenSource();
    var progreso = new Progress<int>(pct => barra.Value = pct);   // corre en la UI
    try
    {
        await Task.Run(() => Importar(ruta, progreso, cts.Token)); // corre en el ThreadPool
        lbl.Text = "Importación terminada";
    }
    catch (OperationCanceledException) { lbl.Text = "Cancelada"; }
};
```

Unidad III · 7. Estados de un hilo

Estado (ThreadState)	Significado	Se llega con
Unstarted	Creado pero no iniciado	new Thread(...)
Running	Ejecutándose o listo para ejecutarse	Start()
WaitSleepJoin	Bloqueado esperando	Sleep(), Join(), lock ocupado
Stopped	Terminó su método	Fin del método del hilo

```
var hilo = new Thread(() => Thread.Sleep(1500)) { Name = "Trabajador" };
Console.WriteLine(hilo.ThreadState);  // Unstarted
hilo.Start();
Thread.Sleep(200);
Console.WriteLine(hilo.ThreadState);  // WaitSleepJoin
hilo.Join();                          // el hilo principal espera
Console.WriteLine(hilo.ThreadState);  // Stopped
```

Unidad III · 8. Mecanismos de sincronización

Cuando dos hilos modifican el mismo dato sin coordinación ocurre una condición de carrera: la operación contador++ en realidad es leer, sumar y escribir, y los hilos pueden intercalarse perdiendo incrementos.

```
object candado = new();
lock (candado) { contador++; }            // sección crítica: un hilo a la vez
```

```
Interlocked.Increment(ref contador2);       // operación atómica

using var semaforo = new SemaphoreSlim(2);  // máximo 2 tareas simultáneas
await semaforo.WaitAsync();
try { /* descarga */ } finally { semaforo.Release(); }
```

Regla de oro de Windows Forms: solo el hilo de la interfaz puede tocar los controles. Desde otro hilo se usa IProgress<T>, Control.Invoke o se devuelve el resultado y se asigna después del await. Violar esta regla produce la excepción «Operación no válida a través de subprocesos».

Unidad III · 9. Detención controlada de hilos

Un hilo nunca debe detenerse a la fuerza (Thread.Abort está obsoleto y no funciona en .NET moderno): puede dejar archivos abiertos o datos a medias. La detención controlada es cooperativa: la interfaz solicita la cancelación con CancellationTokenSource.Cancel() y el hilo revisa el token periódicamente, libera recursos y termina.

```
using var tx = cn.BeginTransaction();
for (int i = 0; i < lineas.Length; i++)
{
    if (token.IsCancellationRequested)
    {
        tx.Rollback();                  // nada queda a medias
        token.ThrowIfCancellationRequested();
    }
    repo.Insertar(Parsear(lineas[i]), cn, tx);
    progreso?.Report((i + 1) * 100 / lineas.Length);
}
tx.Commit();
```

10. Integración: AgroStock v2

```
AgroStock.E02/
├── Data/Db.cs               ← conexión y creación de la base
├── Data/ProductoRepository.cs  ← CRUD parametrizado
├── Models/Producto.cs
└── Forms/
    ├── FrmPrincipal.cs        ← DataGridView + BindingSource
    ├── FrmProducto.cs         ← ComboBox enlazado a categorías
    └── FrmImportar.cs         ← Task.Run, IProgress, CancellationToken
agrostock.sql · productos_importar.csv · DemoHilos/ (consola)
```

Ejercitación

Guía de consultas SQL (individual). Ejecuta agrostock.sql en DB Browser for SQLite y escribe la consulta para:

1. Listar todos los productos ordenados por precio de mayor a menor.
2. Mostrar código, nombre y existencia de los productos con existencia menor que 10.
3. Mostrar los productos que vencen antes del 31/12/2026.
4. Mostrar cada producto con el nombre de su categoría (JOIN).
5. Contar cuántos productos hay por categoría.
6. Calcular el valor total del inventario (precio × existencia).
7. Insertar una nueva categoría «Riego» y un producto de esa categoría.

8. Aumentar 5 % el precio de los fertilizantes.
9. Registrar una SALIDA de 3 unidades del producto HER-004 en movimientos.
10. Eliminar los productos con existencia 0 de la categoría Herramientas (primero verifica con SELECT).

Retos de programación

1. Nivel 1: agrega a AgroStock v2 un ComboBox para filtrar por categoría.
2. Nivel 2: crea MovimientoRepository y un formulario para registrar entradas y salidas que actualice la existencia en la misma transacción.
3. Nivel 3: en DemoHilos, crea una demo productor–consumidor con BlockingCollection<T>.

III. Cierre y consolidación del aprendizaje

Síntesis

- Los datos viven en la base, no en la memoria: sobreviven al cierre y a los apagones.
- Toda consulta con datos del usuario se parametriza.
- El hilo de la interfaz solo dibuja y responde; el trabajo pesado va a otro hilo o tarea.
- La sincronización protege los datos compartidos y la cancelación cooperativa detiene sin dejar datos a medias.

Aplicación (trabajo interencuentro – BOA 2)

Trabajo interencuentro de la BOA 2 (parejas, antes del E3):

1. Corregir AgroStock v1 con la retroalimentación del corte 1.
2. AgroStock v2: CRUD sobre SQLite y tabla de movimientos.
3. Importación en segundo plano con progreso, cancelación y rollback; video de 2 minutos.
4. Informe individual: Thread vs. Task vs. BackgroundWorker con un ejemplo propio.

Complementación

- Microsoft Learn: «Microsoft.Data.Sqlite: información general» – https://learn.microsoft.com/es-es/dotnet/standard/data/sqlite/
- SQLite: «SQL As Understood By SQLite» – https://www.sqlite.org/lang.html
- Microsoft Learn: «Programación asincrónica con async y await» – https://learn.microsoft.com/es-es/dotnet/csharp/asynchronous-programming/
- Microsoft Learn: «Cancelación en subprocesos administrados» – https://learn.microsoft.com/es-es/dotnet/standard/threading/cancellation-in-managed-threads
- Investiga: ¿qué otros motores de base de datos se usan en empresas de Matagalpa? ¿Por qué?

Autoevaluación

Responde sin mirar el folleto y luego verifica con la clave al final. También puedes hacerla en línea en el sitio del curso.

1. ¿Qué sentencia SQL corresponde a la operación Read del CRUD?

a) INSERT
b) SELECT
c) UPDATE
d) DELETE

2. ¿Por qué se usan parámetros ($f) en lugar de concatenar texto en SQL?

a) Porque son más cortos
b) Para evitar la inyección SQL y errores de formato
c) Porque SQLite no acepta comillas
d) No hay diferencia

3. ¿Qué método de SqliteCommand se usa para un INSERT sin resultados?

a) ExecuteReader
b) ExecuteNonQuery
c) ExecuteScalar
d) Read

4. ¿Por qué una ventana Windows Forms «no responde» durante una tarea larga?

a) Porque falta memoria
b) Porque el hilo de la interfaz está ocupado y no procesa eventos
c) Porque la base está cerrada
d) Por un virus

5. Un hilo que ejecuta Thread.Sleep(1000) está en el estado:

a) Unstarted
b) Running
c) WaitSleepJoin
d) Stopped

6. Dos hilos hacen contador++ un millón de veces cada uno sin lock. El resultado final:

a) Siempre es 2 000 000
b) Puede ser menor que 2 000 000
c) Siempre es 1 000 000
d) Lanza una excepción

7. ¿Cuál es la forma correcta de detener una tarea en .NET?

a) Thread.Abort()
b) Cerrar la aplicación
c) Solicitar la cancelación con CancellationToken y que la tarea la revise
d) Poner el hilo en null

Lista de verificación: marca lo que ya lograste.

- ☐ Ejecuté agrostock.sql en DB Browser for SQLite
- ☐ Resolví las 10 consultas SQL

- ☐ Implementé ProductoRepository con parámetros
- ☐ El CRUD funciona desde la interfaz
- ☐ Ejecuté DemoHilos y observé la condición de carrera
- ☐ La importación no congela la ventana
- ☐ Puedo cancelar la importación y se revierte
- ☐ Entregué AgroStock v2, el video y el informe

Glosario

Término	Definición
Clave primaria (PK)	Campo que identifica de forma única cada fila de una tabla.
Clave foránea (FK)	Campo que referencia la clave primaria de otra tabla y crea la relación.
CRUD	Create, Read, Update, Delete: las cuatro operaciones básicas sobre datos.
Inyección SQL	Ataque que altera una consulta concatenando texto malicioso; se evita con parámetros.
Transacción	Grupo de operaciones que se confirman todas (Commit) o ninguna (Rollback).
Hilo	Secuencia de ejecución dentro de un proceso; varios hilos comparten memoria.
Condición de carrera	Error que aparece cuando varios hilos modifican un dato sin sincronización.
lock	Instrucción de C# que permite a un solo hilo a la vez ejecutar una sección crítica.
CancellationToken	Señal para solicitar la detención cooperativa de una tarea.

Referencias

Bibliografía del programa:

- Ceballos Sierra, F. J. Enciclopedia de Microsoft Visual C#. Editorial RA-MA.
- Deitel, H. M. y Deitel, P. J. C# Cómo programar. Editorial Pearson, Prentice Hall.

Documentación complementaria (propuesta por el docente):

- Microsoft (2024). Guía de escritorio (Windows Forms .NET). Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/desktop/winforms/
- Microsoft (2024). Microsoft.Data.Sqlite: información general. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/standard/data/sqlite/
- Microsoft (2024). Subprocesamiento administrado y programación asincrónica. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/standard/threading/
- Microsoft (2024). Entity Framework Core. Microsoft Learn. https://learn.microsoft.com/es-es/ef/core/
- Microsoft (2024). Realización de solicitudes HTTP con la clase HttpClient. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/fundamentals/networking/http/httpclient
- SQLite Consortium (2024). SQL As Understood By SQLite. https://www.sqlite.org/lang.html
- QuestPDF (2024). Documentación. https://www.questpdf.com/
- ClosedXML (2024). Documentación. https://docs.closedxml.io/

Clave de la autoevaluación

- 1) b – SELECT lee datos sin modificarlos.
- 2) b – El valor del parámetro se envía por separado y nunca se interpreta como código SQL.
- 3) b – ExecuteNonQuery devuelve el número de filas afectadas.
- 4) b – Si el hilo de la UI ejecuta el trabajo pesado, no puede redibujar ni atender al usuario.
- 5) c – Sleep, Join y la espera de un lock llevan al estado WaitSleepJoin.
- 6) b – contador++ no es atómica: los hilos se intercalan y se pierden incrementos (condición de carrera).
- 7) c – La cancelación cooperativa permite liberar recursos y revertir la transacción.

Nota: material elaborado con apoyo de inteligencia artificial para la organización y redacción inicial; revisado por el docente.